The dependent recursion principle for the flattened total space of a recursively defined family

The Univalent Foundations Program

hott-book:hits.tex:local-snapshot-20261011; CC BY-SA 3.0 Unported

Editorial note. The counted statement is the author's 'thm:flattening-rect' (source lines 1870-1877, Lemma 6.12.4); the proof environment starts on the next line, so the pairing is by adjacency.

Editorial note. Two context spans are retained: the author's setup defining C, D, the recursive family P and the type W-tilde, and the author's own auxiliary lemma 'thm:flattening-cp' producing c-tilde' and p-tilde', which the counted lemma quantifies over.

Editorial note. The retained counter values (chapter 6, section 12, theorem offset 2) make the retained auxiliary lemma print as Lemma 6.12.3 and the counted lemma as Lemma 6.12.4, matching the original book numbering; theorem-class environments share one counter through aliascnt.

Editorial note. The three external references are navigation pointers to results of other chapters and of the same section that the retained proof invokes by label; the counted proof is retained in full and nothing is replaced by them.

Editorial scope. One original result of the HoTT Book, counted once: the dependent recursion principle for the flattened total space, Lemma 6.12.4.

Allowed standard prerequisites (editorial).

- The flattening setup. Given f,g : B -> A, the coequalizer W generated by c : A -> W and p : prd b:B (c(f b) = c(g b)), a family C : A -> type and a family of equivalences D : prd b:B (C(f b) ~ C(g b)), the family P : W -> type is defined recursively by P(c a) := C a and map P (p b) := ua(D b).. The whole lemma is stated over this setup and the retained context is the author's own definition of it.
- The constructors of W-tilde. W-tilde is generated by c-tilde(a,x) and p-tilde(b,y), and the auxiliary lemma produces c-tilde' into the total space Sigma w:W P(w) together with p-tilde' with its coherence path.. The lemma is about a section of a family Q over that total space, so both constructors are inputs to the statement.
- Dependent paths over the constructors. To define a section of prd x:P(w) Q(w,x) it suffices to give the values and, for each path constructor, a

dependent path; the relevant reformulation is thm:dpath-forall.. It is invoked at source line 1885 to turn the goal into a pointwise equation over y : C(f b).

Source reference. Original lemma 6.12.1: sources/hott-hits/pinned-original-source.tex, source lines 1772–1772. Original result cited by the retained text but located outside the retained span. It is a navigation pointer: the counted statement's own author proof is retained line for line, and no part of it is replaced by this reference.

Source reference. Original lemma 2.3.10: sources/hott-hits/book-labelnumbers-first-edition.tex, source lines 1894–1894. Original result cited by the retained text but located outside the retained span. It is a navigation pointer: the counted statement's own author proof is retained line for line, and no part of it is replaced by this reference.

Source reference. Original lemma 2.9.7: sources/hott-hits/book-labelnumbers-first-edition.tex, source lines 1765–1765. Original result cited by the retained text but located outside the retained span. It is a navigation pointer: the counted statement's own author proof is retained line for line, and no part of it is replaced by this reference.

Now suppose in addition that

- $C : A \to \mathcal{U}$ is a family of types over A, and
- $D : \prod_{(b:B)} C(f(b)) \simeq C(g(b))$ is a family of equivalences over B.

Define a type family $P : W \to \mathcal{U}$ recursively by

$$P(\mathsf{c}(a)) :\equiv C(a)$$
$$P(\mathsf{p}(b)) \coloneqq \mathsf{ua}(D(b)).$$

Let $\widetilde{W}$ be the higher inductive type generated by

- $\widetilde{\mathsf{c}} : \prod_{(a:A)} C(a) \to \widetilde{W}$ and
- $\widetilde{\mathsf{p}} : \prod_{(b:B)} \prod_{(y:C(f(b)))} (\widetilde{\mathsf{c}}(f(b), y) =_{\widetilde{W}} \widetilde{\mathsf{c}}(g(b), D(b)(y)))$.

The idea is to show that $\sum_{(x:W)} P(x)$ has the same universal property as $\widetilde{W}$. We begin by showing that it comes with analogues of the constructors $\widetilde{\mathsf{c}}$ and $\widetilde{\mathsf{p}}$.

Lemma 6.12.3. *There are functions*

- $\widetilde{\mathsf{c}}' : \prod_{(a:A)} C(a) \to \sum_{(x:W)} P(x)$ *and*
- $\widetilde{\mathsf{p}}' : \prod_{(b:B)} \prod_{(y:C(f(b)))} \left(\widetilde{\mathsf{c}}'(f(b), y) =_{\sum_{(w:W)} P(w)} \widetilde{\mathsf{c}}'(g(b), D(b)(y))\right)$.

Lemma 6.12.4. *Suppose* $Q : \left(\sum_{(x:W)} P(x)\right) \to \mathcal{U}$ *is a type family and that we have*

- $c : \prod_{(a:A)} \prod_{(x:C(a))} Q(\widetilde{\mathsf{c}}'(a, x))$ *and*
- $p : \prod_{(b:B)} \prod_{(y:C(f(b)))} \left(\widetilde{\mathsf{p}}'(b, y)_*(c(f(b), y)) = c(g(b), D(b)(y))\right)$.

Then there exists $k : \prod_{(z:\sum_{(w:W)} P(w))} Q(z)$ *such that* $k(\widetilde{\mathsf{c}}'(a,x)) \equiv c(a,x)$.

Proof. Suppose given $w : W$ and $x : P(w)$; we must produce an element $k(w,x) : Q(w,x)$. By induction on w, it suffices to consider two cases. When $w \equiv \mathsf{c}(a)$, then we have $x : C(a)$, and so $c(a,x) : Q(\mathsf{c}(a),x)$ as desired. (This part of the definition also ensures that the stated computational rule holds.)

Now we must show that this definition is preserved by transporting along $\mathsf{p}(b)$ for any $b : B$. Since what we are defining, for all $w : W$, is a function of type $\prod_{(x:P(w))} Q(w,x)$, by original lemma 2.9.7 it suffices to show that for any $y : C(f(b))$, we have

$$\mathsf{transport}^Q(\mathsf{pair}^=(\mathsf{p}(b), \mathsf{refl}_{\mathsf{p}(b)_*(y)}), c(f(b),y)) = c(g(b), \mathsf{p}(b)_*(y)).$$

Let $q : \mathsf{p}(b)_*(y) = D(b)(y)$ be the path obtained from original lemma 6.12.1. Then we have

$$\begin{aligned} c(g(b), \mathsf{p}(b)_*(y)) &= \mathsf{transport}^{x\mapsto Q(\widetilde{\mathsf{c}}'(g(b),x))}(q^{-1}, c(g(b), D(b)(y))) \\ &\qquad\qquad \text{(by } \mathsf{apd}_{x\mapsto c(g(b),x)}(q^{-1})^{-1}\text{)} \\ &= \mathsf{transport}^Q(\mathsf{ap}_{x\mapsto\widetilde{\mathsf{c}}'(g(b),x)}(q^{-1}), c(g(b), D(b)(y))). \\ &\qquad\qquad \text{(by original lemma 2.3.10)} \end{aligned}$$

Thus, it suffices to show

$$\mathsf{transport}^Q\Big(\mathsf{pair}^=(\mathsf{p}(b), \mathsf{refl}_{\mathsf{p}(b)_*(y)}),\ c(f(b),y)\Big) = \mathsf{transport}^Q\Big(\mathsf{ap}_{x\mapsto\widetilde{\mathsf{c}}'(g(b),x)}(q^{-1}),\ c(g(b),D(b)(y))\Big).$$

Moving the right-hand transport to the other side, and combining two transports, this is equivalent to

$$\mathsf{transport}^Q\Big(\mathsf{pair}^=(\mathsf{p}(b), \mathsf{refl}_{\mathsf{p}(b)_*(y)}) \cdot \mathsf{ap}_{x\mapsto\widetilde{\mathsf{c}}'(g(b),x)}(q),\ c(f(b),y)\Big) = c(g(b),D(b)(y)).$$

However, we have

$$\begin{aligned} &\mathsf{pair}^=(\mathsf{p}(b), \mathsf{refl}_{\mathsf{p}(b)_*(y)}) \cdot \mathsf{ap}_{x\mapsto\widetilde{\mathsf{c}}'(g(b),x)}(q) = \\ &\qquad \mathsf{pair}^=(\mathsf{p}(b), \mathsf{refl}_{\mathsf{p}(b)_*(y)}) \cdot \mathsf{pair}^=(\mathsf{refl}_{\mathsf{c}(g(b))}, q) = \mathsf{pair}^=(\mathsf{p}(b), q) = \widetilde{\mathsf{p}}'(b,y) \end{aligned}$$

so the construction is completed by the assumption $p(b,y)$ of type

$$\mathsf{transport}^Q(\widetilde{\mathsf{p}}'(b,y), c(f(b),y)) = c(g(b),D(b)(y)). \qquad \square$$